



Advanced Database Design - Fall 2026

1. Course Information

Contact Information

- **Instructor:** Dr. Terry Griffin
- **Office:** Bolin Hall 124-F
- **Office Hours:** MW: 2:30 p.m. – 4:00 p.m.; TTh: 11:00 a.m. – 12:00 p.m.
- **Office Phone:** (940) 397-4439
- **Email:** terry.griffin@msutexas.edu

Important Dates, Times, and Location

- **Course Number / Section:** CMPS 5303, Section 101
- **Days:** Monday / Wednesday
- **Time:** 4:00 p.m. – 5:20 p.m.
- **Location:** Bolin 248
- **Semester Start:** Monday, August 23
- **Semester End:** Friday, December 4
- **Labor Day:** Monday, September 7
- **Thanksgiving Holiday:** Wednesday, November 25 – Sunday, November 29
- **Last Day for "W":** Monday, November 23
- **Final Exam:** Monday, December 7, 3:30 p.m. – 5:30 p.m.

Course Communication

- **Zoom:** <https://msutexas-edu.zoom.us/j/9403974439>
- **GitHub:** Course repository information will be provided in class.

Course announcements, assignment information, questions, and course discussion may be distributed through the course communication platform and GitHub.

Textbook and Course Materials

Required Textbook: None

No textbook is required. Course materials will be provided by the instructor or made available through course resources.

Students will need access to a computer capable of running the database systems and development tools introduced during the semester, including Git and GitHub.

Prerequisites and Expected Background

This is a graduate-level course. Students should enter with prior programming experience and a working familiarity with data structures, software development tools, and basic database concepts.

Students should be prepared to:

- write and debug programs in at least one modern programming language;
 - use a command line, Git, and GitHub;
 - design and work with structured data; and
 - read technical documentation and conduct independent technical investigation.
-

2. Course Overview

Course Description

Modern applications rarely have the luxury of asking simply, “Which SQL database should we use?” Developers choose among relational databases, document stores, key-value systems, graph databases, search engines, distributed databases, and systems that combine features from several of these categories.

This course is a survey of modern database systems. Its emphasis is on understanding why different database models exist, how they differ, and how to select an appropriate system for a particular application, workload, and set of requirements.

Rather than attempting shallow coverage of a large number of products, the course studies representative systems from several major database families in enough depth to build applications, conduct experiments, and compare their behavior.

The primary systems are:

- **PostgreSQL** — primary relational database management system;
- **SQLite** — embedded relational database;
- **MongoDB** — primary document-oriented database; and
- **Redis** — primary key-value and in-memory data store.

Additional systems may be used for comparison, presentations, or projects.

Course Approach

Traditional database courses often devote substantial time to relational algebra, relational calculus, normalization theory, and formal relational database design. Those foundations matter, and this course covers the relational concepts needed to understand modern relational systems: relations, keys, normalization, joins, constraints, transactions, and query processing.

This course is not primarily a course in formal relational database theory. Its emphasis is on the design, implementation, use, evaluation, and comparison of modern database systems.

The central question of the course is:

Given an application and workload, what database architecture should we use, and why?

There is no universally best database. Students will connect theory to engineering decisions by evaluating tradeoffs among correctness, consistency, performance, scalability, operational complexity, and developer convenience.

3. Learning Objectives

By the end of the course, students should be able to:

1. Explain the major characteristics of relational, document-oriented, and key-value database models.
2. Design and implement databases using multiple data models.
3. Write effective queries using SQL and non-SQL query interfaces.
4. Explain keys, relationships, normalization, joins, referential integrity, and schema design.
5. Design and evaluate indexes for different workloads.
6. Explain transactions, ACID properties, concurrency, isolation, and consistency guarantees across database systems.
7. Compare approaches to replication, partitioning, caching, and distributed storage.
8. Measure database behavior using reproducible performance experiments.
9. Evaluate tradeoffs among consistency, performance, scalability, complexity, and developer convenience.
10. Select an appropriate database technology for a given application and justify the decision with technical evidence.
11. Communicate technical findings through presentations, documentation, and reproducible software projects.

4. Course Content

Major Course Themes

Database Models and Data Modeling

Students will compare relational, document, key-value, and selected specialized models. Topics include:

- relations, tables, documents, collections, keys, and values;
- keys, relationships, constraints, and referential integrity;
- normalization and deliberate denormalization;
- embedded documents versus references;
- schema enforcement and schema flexibility; and
- data duplication and consistency.

Querying and Data Access

Students will use and compare query models appropriate to each system.

- SQL, joins, aggregation, views, and query plans;
- document queries and MongoDB aggregation pipelines;
- key-based access and data-structure operations;
- query APIs; and
- selected search and specialized query models.

Indexing, Performance, and Benchmarking

Topics include B-tree and other indexing approaches, selectivity, query planning, execution plans, read/write tradeoffs, and memory versus disk access.

Students will conduct reproducible experiments involving such measures as latency, throughput, updates, bulk operations, concurrent operations, storage, cache behavior, and scaling. Experimental work must document the environment, dataset, configuration, workload, indexes, trials, measurements, methodology, and limits of the conclusions.

Transactions, Concurrency, and Consistency

Topics include transactions; atomicity, consistency, isolation, and durability; locking; multi-version concurrency control; isolation levels; distributed consistency; and eventual consistency.

ACID will be treated as a set of guarantees to investigate, not as a binary label. Different systems provide different transaction capabilities, scopes, costs, and consistency guarantees.

Distributed and Specialized Systems

Selected topics may include replication, partitioning, distributed storage, wide-column databases, graph databases, search systems, cloud and serverless databases, and vector search.

Database Selection

The course repeatedly returns to database selection: identifying the workload, requirements, tradeoffs, operational costs, and competing solutions that inform a defensible architecture decision.

Core Database Systems

PostgreSQL

PostgreSQL is the primary relational system. Topics may include schema design, SQL, constraints, transactions, indexing, query planning, views, concurrency, JSON/JSONB, full-text search, extensions, and selected advanced capabilities.

SQLite

SQLite provides a contrast to client/server relational systems. Topics may include embedded and file-based databases, zero-configuration deployment, transaction behavior, concurrency characteristics, and appropriate workloads.

MongoDB

MongoDB is the primary document store. Topics may include BSON documents, collections, flexible schemas, embedding versus references, indexing, aggregation pipelines, replication, transactions, and comparative data modeling. A recurring comparison is MongoDB documents versus PostgreSQL JSON/JSONB.

Redis

Redis is the primary key-value and in-memory system. Topics may include keys, expiration, caching, strings, lists, sets, sorted sets, hashes, persistence, transactions, pub/sub, streams, and distributed caching.

Supporting / Specialized Topics

Representative systems may be examined through lectures, presentations, or projects.

- Wide-column databases: Cassandra, ScyllaDB, or HBase.
- Graph databases: Neo4j and graph-oriented workloads.
- Search systems: Elasticsearch or OpenSearch.
- Vector search: PostgreSQL with **pgvector**, Redis vector search, and selected vector databases.
- Comparative systems: Valkey, DynamoDB, Firestore, Couchbase, or other systems appropriate to current course work.

Tentative Course Progression

The exact sequence and pace may change based on class progress, projects, student questions, and developments in database technology.

Part I — Database Foundations

- database models and data modeling;
- relational concepts, SQL, keys, and relationships;
- normalization, denormalization, and constraints;
- indexing, query planning, transactions, and ACID.

Part II — Relational Systems

- PostgreSQL and SQLite;
- schema design, query execution, indexing, and performance;
- concurrency and transaction behavior.

Part III — Document Systems

- MongoDB;
- documents versus normalized relations;
- MongoDB versus PostgreSQL JSON/JSONB;
- schema flexibility versus schema enforcement.

Part IV — Key-Value and In-Memory Systems

- Redis;
- caching and fast access patterns;
- Redis as cache versus primary data store;
- persistent versus memory-oriented storage.

Part V — Distributed and Specialized Systems

- replication, partitioning, and distributed consistency;
- wide-column, graph, search, cloud, serverless, and vector systems.

Part VI — Database Selection and Evaluation

- comparative experiments;
- technical presentations;
- database architecture selection and justification.

5. Coursework and Assessment

Presentations

Each student will give **two technical presentations** during the semester. Topics will be assigned or approved by the instructor and will generally involve a database technology, architecture, feature, or current development.

Presentations should move beyond a feature list. They should address the problem a technology solves, its data model, favorable and unfavorable workloads, guarantees, performance characteristics, operational costs, competing technologies, and the reasons a developer might choose it.

Programming Projects

Programming projects require students to build, query, measure, or compare database systems. Work may include constructing datasets, implementing equivalent schemas across models, generating workloads, benchmarking, indexing experiments, transaction or concurrency experiments, caching, API development, and comparative analysis.

Projects must be reproducible. At a minimum, each submission should include a README, setup and execution instructions, dependency information, database setup or schema definitions, meaningful documentation, and a discussion of results.

Final Examination

The final examination assesses students' ability to apply the course's comparative framework to database systems, workloads, guarantees, tradeoffs, and technology-selection decisions. It may include conceptual analysis and interpretation of technical scenarios.

GitHub Portfolio

Students will maintain a GitHub repository containing their course work. The portfolio should be organized, readable, and reproducible, and may include source code, setup scripts, schema definitions, sample data or generators, queries, experiment scripts, results, documentation, and presentation materials.

Credentials, passwords, private keys, and other sensitive information must never be committed to GitHub.

6. Grading

Grade Distribution

Category	Weight
Presentations	20%
Programming Projects	40%
Final Examination	20%
GitHub Portfolio	20%
Total	100%

Grade Scale

Grade	Percentage
A	90–100
B	80–89
C	70–79
D	60–69
F	Below 60

Grading Notes

Grades reflect the quality, completeness, technical soundness, reproducibility, and documentation of submitted work, as applicable to each assessment.

7. Common Course Policies

Participation

This course depends on discussion, experimentation, troubleshooting, and comparison of results. Meaningful participation includes engaging with course work, asking and answering technical questions,

discussing experimental results, and contributing professionally to the learning environment.

Course Delivery

Course materials, announcements, assignments, and supplemental resources may be provided through the course communication platform, GitHub, or other instructor-designated resources. Students are responsible for checking these resources regularly.

Assignment and Submission Requirements

Unless an assignment states otherwise, submitted work must be complete, readable, and submitted through the method and by the deadline specified by the instructor. Work submitted in an incorrect location or format may be treated as not submitted.

Program Execution and Documentation Requirements

Programs and technical projects must include the instructions, dependencies, configuration information, and documentation needed for the instructor to evaluate the work. A submission that cannot be run or understood cannot provide reliable evidence that it works.

GitHub and Repository Requirements

Students are responsible for maintaining repositories in the form required for the course. Repositories should be organized and include appropriate documentation. Do not commit secrets, credentials, private keys, or other sensitive information.

Understanding and Oral Defense of Submitted Work

Students must be able to explain submitted work, including design decisions, code, results, and documentation. The instructor may ask students to demonstrate or discuss their work. Inability to explain a submission may affect the grade and may require further review.

Use of Large Language Models and AI-Assisted Tools

AI-assisted tools may be used only as permitted for a specific assignment. Students remain responsible for understanding, testing, documenting, and being able to explain all submitted work. Submitting work that a student cannot explain as their own is not acceptable.

Attendance

Students are expected to attend class and participate in course activities. Attendance does not replace responsibility for completing all course work and keeping up with announcements and deadlines.

Classroom Conduct

Professional, respectful conduct is expected. Conduct that disrupts instruction or prevents others from participating productively is not acceptable.

Computer and Internet Requirements

Students must maintain reliable access to a computer and internet connection suitable for the course. They are responsible for backing up work and planning for ordinary technical problems.

Late Work

Late-work rules, including any assignment-specific exceptions, will be stated with the assignment or by the instructor. Students should communicate promptly when circumstances may affect a deadline.

Missed Exams and Quizzes

Students who miss an examination or quiz must contact the instructor as soon as possible. Make-up work, when permitted, is at the instructor's discretion and may differ in format or scope.

Final Examination Scheduling

The final examination is administered at the university-scheduled time listed in this syllabus. Requests involving final-exam conflicts must follow applicable university procedures and be communicated promptly.

Testing Procedures

Students must follow all instructions for examinations and assessments. Unauthorized materials, assistance, communication, or devices are prohibited.

Academic Collaboration

Collaboration is permitted only to the extent stated for a particular assignment. Students may discuss ideas and course concepts, but submitted work must accurately represent each student's own effort unless group work is expressly assigned.

Recording of Classes

Students may not record class activities without the instructor's permission and any required consent. Approved recordings may be used only for authorized educational purposes.

8. Department Resources and Policies

Computer Science Tutoring

Students are encouraged to use available Computer Science tutoring and instructional-support resources. Availability, locations, and schedules will be communicated by the department or instructor.

Department Testing Policy

Department and university testing procedures apply to course examinations. Students should follow instructor directions and any required testing-center procedures.

Department Programming Assignment Policy

Programming assignments must reflect the student's own work except where collaboration or reuse is expressly authorized. Students must be prepared to explain their work and cite any permitted external resources.

Department Academic Misconduct Procedures

Suspected academic misconduct may be handled under department and university procedures in addition to any academic consequences for the assignment or course.

9. University Policies and Resources

Academic Misconduct

Academic dishonesty—including plagiarism, unauthorized collaboration, fabrication, and use of unauthorized assistance—is prohibited. Allegations will be addressed under current university policy.

Students Requiring Accommodations

Students who require academic accommodations should contact the university office responsible for disability support and provide the instructor with the appropriate documentation as early as possible. Reasonable accommodations are provided in accordance with university policy.

Midterm Progress Reports

Midterm progress reports will be submitted when required by university policy. Students should monitor their standing throughout the semester and contact the instructor promptly with questions.

Campus Carry

Campus-carry policies and applicable university regulations apply to this course. Students should consult current university guidance for details.

Tobacco Policy

University tobacco and smoke-free-campus policies apply to all course activities and facilities.

Moffett Library

Moffett Library provides research assistance, databases, technology resources, and other academic support services. Students are encouraged to use these resources for course research and technical investigation.

Student Technical Support

Students needing assistance with university technology services should use current university technical-support resources.

Tutoring and Academic Support

The university provides tutoring, academic-support, and student-success resources. Students are encouraged to seek assistance early when they need it.

University policies and services may change. Current official university policy takes precedence over this summary.